

Unitree Week 30

Presentation by Mario Ortiz

This Week

- Preparing to implement Unitree R1 flat locomotion environment simulation onto physical robot.

Deploying RL models onto the Physical Unitree R1

- Isaac Lab (R1 model, flat_env_cfg.py, PPO RSL-RL)
- Trained checkpoint model_xyz.pt file
- Export trained checkpoint: becomes policy.onnx/policy.pt
- Sim-to-Sim validation (bring policy.onnx to MuJoCo for example and test there)
- R1 deployment controller:
 - IMU
 - Joint Positions
 - Joint Velocities
 - Joystick Velocity Command
- Unitree SDK2 Low Command

Deploying RL models onto the Physical Unitree R1

MJLab R1 Policy

83 Observations [vx, vy, yaw_rate. etc.]:
Actor Neural Network: 83
→ 512 → 256 → 128 →
24 policy actions

Exporting Deployment Policy

Export Deployment Policy:
model_xxxx.py →
MJLab/RSL-RL export →
**policy.onnx and
policy.onnx.data**

Deployment
Configuration:

- deploy.yaml
- config.yaml

R1 Deployment Controller

Unitree LowState:

- IMU
 - Joint Position q
 - Joint velocity dq
 - Robot state
- Deployment
Observation Manager

ONNX Inference

83 Observations →
policy.onnx → 24 actions

Action Processing:

Raw policy action
↓
Action scaling
↓
Default pose
↓
Desired Joint Position

R1 Joint Mapping:

Policy joints 0-23 →
joint_ids_map → Physical
R1 Motor Indices

Unitree LowCmd (SKD2)

- Controller Supplies PD gain inputs
- Policy Outputs 24 normalized actions → Action processing → q_des → LowCmd

Command Input

- Ex. Keyboard, ROS node
- (could also make our own application)

Sim2Real Transition Controller

↓
DDS via Ethernet
↓
R1

MJLab

- MJLab requires the use of Linux/Ubuntu rather than Windows.
 - Will attempt this setup via Windows Subsystem Linux
 - In addition, setup for wslg (Windows Subsystem Linux GUI) for visual interface
- MJlab controls 24 actuator joints
 - Doesn't control the 2 head joints
 - Doesn't control the 14 hand joints

MJLab setup

- Confirm NVIDIA gpu: “nvidia-smi”
 - This confirms if your device has an NVIDIA GPU
- echo “\$DISPLAY”
- echo “WAYLAND_DISPLAY”
 - These two confirm that WSLg is active. WSLg allows Linux GUI applications such as MuJoCO’s native viewer to appear as normal Windows desktop windows.
- glxinfo -B
 - OpenGL renderer; this shows what GPU WSLg is using for OpenGL rendering.
 - In my case, my laptop has two gpu’s so it reads from the second GPU (AMD) → therefore I must initiate commands by setting GPU to NVIDIA.
- MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA glxinfo -B
 - This forces WSLg OpenGL onto the NVIDIA GPU (particularly, my RTX 3080)
- MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA glxgears
 - GUI Test; a test to see that an actual Linux graphical window could open through WSLg while using the NVIDIA gpu.

MJLAB System Dependencies Setup

- `sudo apt update`
 - Updates Ubuntu's package lists after changing repository configuration
- Install Required Linux development packages

```
sudo apt install -y \  
    git \  
    libyaml-cpp-dev \  
    libboost-all-dev \  
    libeigen3-dev \  
    libspdlog-dev \  
    libfmt-dev
```

These provide Git plus common C++ development libraries needed by the Unitree MuJoCo deployment ecosystem

- `conda --version`
 - If “Command ‘conda’ not found”, need to install conda
-

Conda Setup

```
mkdir -p ~/miniconda3
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
  -O ~/miniconda3/miniconda.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
~/miniconda3/bin/conda init --all
source ~/.bashrc
conda tos accept --override-channels \
  --channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels \
  --channel https://repo.anaconda.com/pkgs/r
```

MJLab Python Environment Setup

```
conda create -n unitree_rl_mjlab python=3.11 -y
conda activate unitree_rl_mjlab      #prompt will become: “(unitree_rl_mjlab) username@LAPTOP:~$”
python --version                    #Python Verification (must output Python 3.11.xx)
which python
which pip
```

Unitree MJLab Installation

```
cd ~
git clone https://github.com/unitreerobotics/unitree_rl_mjlab.git
cd unitree_rl_mjlab
ls
pwd
pip install -e .

#obtained different/missing working versions of mujoco and warp-lang so must do this command:
pip install --force-reinstall \
    "mujoco==3.6.0" \
    "warp-lang==1.12.0"

pip install "scipy>=1.15,<2"          # Missing SciPy dependency
pip check
```

Unitree MJLab Training

Verifying MJLab setup: use the --help command:
`python scripts/train.py Unitree-R1-Flat --help`

Small Training Test:
`python scripts/train.py Unitree-R1-Flat \`
 `--env.scene.num-envs=256 \`
 `--agent.max-iterations=100 \`
 `--agent.save-interval=25 \`
 `--agent.logger tensorboard \`
 `--agent.run-name wsl_smoke_test`

Play Training Checkpoints:
`MESA_D3D12_DEFAULT_ADAPTER_NAME=NVID`
`IA \`
`python scripts/play.py Unitree-R1-Flat \`
 `--checkpoint-file`
 `logs/rsl_rl/r1_velocity/2026-08-29_11-54-25_wsl_smoke`
 `_test/model_99.pt \`
 `--num-envs 1 \`
 `--viewer native`

Unitree MJLab Training

Play Training Checkpoints:

```
MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA \
```

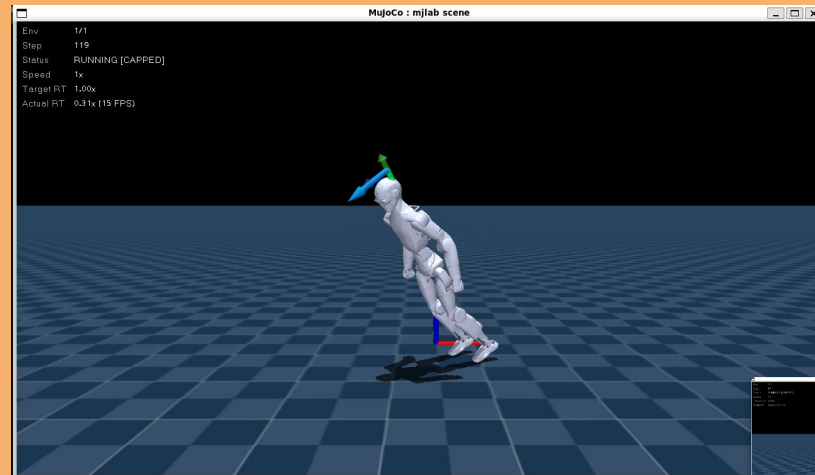
```
python scripts/play.py Unitree-R1-Flat \
```

```
--checkpoint-file
```

```
logs/rsl_rl/r1_velocity/2026-08-29_11-54-25_wsl_smoke  
_test/model_99.pt \
```

```
--num-envs 1 \
```

```
--viewer native
```



Unitree MJLab Training

```
python scripts/train.py Unitree-R1-Flat \  
--env.scene.num-envs=4096 \  
--agent.max-iterations=100 \  
--agent.logger tensorboard \  
--agent.run-name r1_4096_benchmark
```

```
#####  
Learning iteration 2018/10001  
  
Total steps: 198475776  
Steps per second: 20679  
Collection time: 4.264s  
Learning time: 0.489s  
Mean value loss: 0.0082  
Mean surrogate loss: -0.0106  
Mean entropy loss: 7.1751  
Mean reward: 42.48  
Mean episode length: 1000.00  
Mean action std: 0.34  
Episode_Reward/track_linear_velocity: 0.7735  
Episode_Reward/track_angular_velocity: 0.6893  
Episode_Reward/body_orientation_l2: -0.0037  
Episode_Reward/pose: 0.7645  
Episode_Reward/body_ang_vel: -0.0180  
Episode_Reward/angular_momentum: -0.0195  
Episode_Reward/is_terminated: 0.0000  
Episode_Reward/joint_acc_l2: -0.0563  
Episode_Reward/joint_pos_limits: -0.0002  
Episode_Reward/action_rate_l2: -0.3465  
Episode_Reward/foot_gait: 0.4419  
Episode_Reward/foot_clearance: -0.0716  
Episode_Reward/foot_slip: -0.0107  
Episode_Reward/soft_landing: -0.0171  
Episode_Reward/stand_still: -0.0027  
Episode_Reward/self_collisions: -0.0020  
Episode_Metrics/mean_action_acc: 0.6822  
Metrics/twist/error_vel_xy: 0.5575  
Metrics/twist/error_vel_yaw: 1.0642  
Episode_Termination/time_out: 3.8750  
Episode_Termination/fell_over: 0.0000  
Metrics/angular_momentum_mean: 0.7637  
Metrics/slip_velocity_mean: 0.0096  
Metrics/landing_force_mean: 195.2629  
  
-----  
Iteration time: 4.75s  
Time elapsed: 01:38:53  
ETA: 06:30:59
```

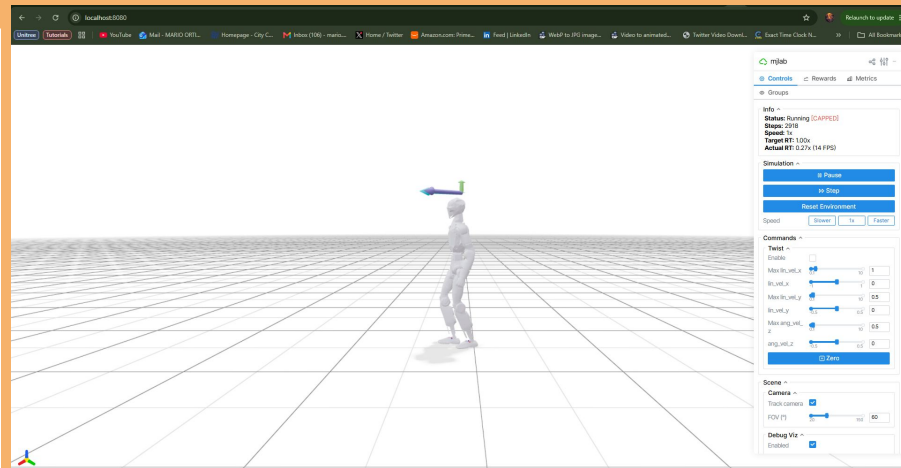
Unitree MJLab Training

Found out the locomotion task has the capability of us choosing specific directions (in the x,y,z plane) for the robot to move to.

One way this could be done is via Viser

It opens a host website with the Unitree R1 doing the locomotion task. We can choose the specific range for x,y, and z velocity as well as the specific x,y, and z velocity we want it moving in.

```
python scripts/play.py Unitree-R1-Flat \  
    --checkpoint-file  
logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/model_200  
0.pt \  
    --num-envs 1 \  
    --viewer viser
```



ONNX File

- On MJLab (and most likely also on IsaacLab), the train function also outputs an .ONNX file, so no conversion is needed (though no .onnx.data file is found; this may be due to it being stored in the .onnx)

```
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ ls -lh logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/model_2000.pt logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/policy.onnx*  
-rw-r--r-- 1 mario mario 4.9M Aug 29 14:29 logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/model_2000.pt  
-rw-r--r-- 1 mario mario 825K Aug 29 14:29 logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/policy.onnx
```

- Next step will be to verify ONNX file parameters: (ie. the policy contract must correspond to 83 actor observations (inputs) with 24 policy actions (outputs))
 - Using a configuration script, I confirmed it satisfies the requirements

```
Inputs:  
obs [1, 83]  
  
Outputs:  
actions [1, 24]
```

- Next step is to create a R1 deployment workspace

R1 Deployment Workspace

- #making and copying .ONNX to R1 redeployment directory
`mkdir -p deploy/robots/r1/config/policy/velocity/v0/exported`
`cp logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/policy.onnx \`
`deploy/robots/r1/config/policy/velocity/v0/exported/policy.onnx`
- Obtaining the deploy .yaml file (already in this folder):
`cat deploy/robots/r1/config/policy/velocity/v0/params/deploy.yaml`
- Obtaining the controller-level R1 config .yaml file
`cat deploy/robots/r1/config/config.yaml`

Setting up the R1 controller:

```
cd ~/unitree_rl_mjlab/deploy/robots/r1
mkdir -p build
cd build
cmake ..
make -j4
```

YAML File

The `deploy.yaml` file is how the RL policy interfaces with the RL

```

unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ cat deploy/robots/r1/config/policy/velocity/v0/params/deploy.yaml
joint_ids_map: [0,1,2,3,4,5,6,7,8,9,10,11,12,13,15,16,17,18,19,22,23,24,25,26]
step_dt: 0.02
stiffness: [100, 100, 100, 100, 100, 40, 40, 100, 100, 100, 100, 40, 40, 100, 100,
            40, 40, 20, 20, 20, 40, 40, 20, 20, 20]
damping: [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
          2, 2, 1, 1, 1, 2, 2, 1, 1, 1]
default_joint_pos: [-0.1,0,0,0.3,-0.2,0,-0.1,0,0,0.3,-0.2,0, 0,0, 0.35,0.18,0,0.87,0, 0.35,-0.18,0,0.87,0]

commands:
  base_velocity:
    ranges:
      lin_vel_x: [-0.5, 1.0]
      lin_vel_y: [-0.5, 0.5]
      ang_vel_z: [-1.0, 1.0]
      heading: null
actions:
  JointPositionAction:
    clip: null
    joint_names: [.*]
    scale: [0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25,
             0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25]
    offset: [-0.1,0,0,0.3,-0.2,0,-0.1,0,0,0.3,-0.2,0, 0,0, 0.35,0.18,0,0.87,0, 0.35,-0.18,0,0.87,0]
    joint_ids: null
observations:
  base_ang_vel:
    params: {}
    clip: null
    scale: [1.0, 1.0, 1.0]
    history_length: 1
  projected_gravity:
    params: {}
    clip: null
    scale: [1.0, 1.0, 1.0]
    history_length: 1
  velocity_commands:
    params: {command_name: base_velocity}
    clip: null
    scale: [1.0, 1.0, 1.0]
    history_length: 1
  gait_phase:
    params: {period: 0.6}
    clip: null
    scale: [1.0, 1.0]
    history_length: 1
  joint_pos_rel:
    params: {}
    clip: null
    scale: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
            1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
    history_length: 1
  joint_vel_rel:
    params: {}
    clip: null
    scale: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
            1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
    history_length: 1
  last_action:
    params: {}
    clip: null
    scale: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
            1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
    history_length: 1

```

Notes on Deploy YAML file

- Physical R1 Motor Mapping:
 - joint_ids_map: [0,1,2,3,4,5,6,7,8,9,10,11,12,13,15,16,17,18,19,22,23,24,25,26]
 - Though we have 24 actions (ONNX), there are 26 joints (2 are the neck); we must ensure we map the ONNX actions with the correct corresponding joint number.
- Control Period: step_dt : 0.02
 - 0.02 control period = 50 Hz → this must match the control period of the trained policy
- PD gains:
 - stiffness: [100, 100, 100, 100, 40, 40, 100, 100, 100, 100, 40, 40, 100, 100,
 - 40, 40, 20, 20, 20, 40, 40, 20, 20, 20]
 - damping: [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
 - 2, 2, 1, 1, 1, 2, 2, 1, 1, 1]
 - K_p = stiffness, K_d = damping for each controlled joint; the controller provides these values from this configuration.
- Default Pose
- Action Processing: Provides information (scale, offset) on how to interpret the neural-network outputs.
- Commands:

Notes on Deploy YAML file

- Action Processing:
 - actions:
 - JointPositionAction:
 - clip: null
 - joint_names: [.*]
 - scale: [0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25]
 - offset: [-0.1,0,0,0.3,-0.2,0, -0.1,0,0,0.3,-0.2,0, 0,0, 0.35,0.18,0,0.87,0, 0.35,-0.18,0,0.87,0]
 - joint_ids: null
 - Provides information (scale, offset) on how to interpret the neural-network outputs.
- Commands: base velocity parameters ([vx, vy, yaw_rate])
- Observations: Provides information for the 83-input ONNX base_ang_vel, projected_gravity, velocity_commands, → these all add up to 83)
 - **This confirms our yaml file can be properly integrated with our .ONNX file**

Config Yaml File

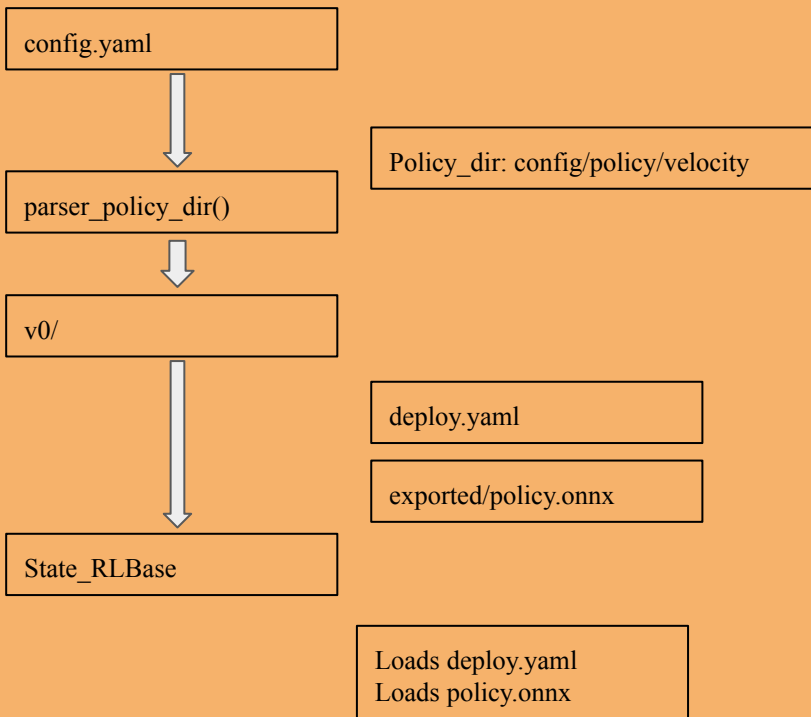
The config.yaml file describes how the robot operates.

```
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ cat deploy/robots/rl/config/config.yaml
FSM:
-: # enabled fsm
  Passive:
    id: 1
  FixStand:
    id: 2
  Velocity:
    id: 3
    type: RLBase
  Passive:
  transitions:
    FixStand: LT + up.on_pressed
    mode: [
      1,1,1,1,1,1,
      1,1,1,1,1,1,
      1,1,1,
      1,1,1,1,1,
      1,1,
      1,1,1,1,
      1,1,1,1,1,
      1,1,1,1,1,
    ]
    kd: [
      3,3,3,3,3,3,
      3,3,3,3,3,3,
      3,3,3,
      3,3,3,3,3,3,3,
      3,3,
      3,3,3,3,3,3,3,
    ]
  FixStand:
  transitions:
    Passive: LT + B.on_pressed
    Velocity: RT + A.on_pressed
    kp: [
      100, 100, 100, 150, 40, 40,
      100, 100, 100, 150, 40, 40,
      200, 200, 200,
      40, 40, 40, 40, 40,
      100, 100,
      40, 40, 40, 40, 40,
    ]
    kd: [
      2,2,2,4,2,2,
      2,2,2,4,2,2,
      5,5,5,
      10,10,10,10,10,
      4,4,
      10,10,10,10,10,
    ]
    ts: [0, 3]
    qs: [
      [],
      [
        -0.1, 0,0,0.3,-0.2,0,
        -0.1, 0,0,0.3,-0.2,0,
        0,0,0,
        0.35, 0.18,0,0.87,0,
        0,0,
        0.35,-0.18,0,0.87,0,
      ]
    ]
  Velocity:
  transitions:
    Passive: LT + B.on_pressed
    policy_dir: config/policy/velocity(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ |
```

Notes on Config YAML file

- “config.yaml “ contains a Finite State Machine, operating in these 3 states:
 - Passive: state where the RL policy is not commanding locomotion
 - FixStand: A middle-ground state; prevents the robot from going straight from startup into neural-network control.
 - Has commands for going to Passive State or Velocity State
 - Velocity: This state is where the RL policy is occurring
 - This state has a type: RLBase, signifying it uses the RL policy infrastructure
- kp and kd: In this file, the kp and kd gains are used for putting the robot into its standing posture.
- ts and qs: These define **FixStand Trajectory**, meaning
- Velocity: Opens RLBase to on → moves to config/policy/velocity → selects policy deployment profile (policy.onnx + deploy.yaml)
-

Deployment Sim / Sim2Sim



- The initial simulation bypasses most real deployment infrastructure.
- The current goal now is to simulate the Unitree R1 using the actual deployment path we intend to use on the robot.

Deployment Sim / Sim2Sim

- The robot scene is made using an XML file.
- **r1.xml** only contains the robot definition (the g1.xml contains the definition as well as actuators, sensors, ground plane, lighting, etc. in its corresponding file).
 - We obtain the R1's actuator from MJLab dynamically using **r1_constants.py**
- We need to set up the r1.xml at the robot_scene similar to the g1's configuration.
- Currently, there are two issues at hand with the R1 configuration.
 - R1 Sim2Sim bug: misalignment with the R1's simulator bridge (treats MuJoCO actuators as DDS slots (0-23), while R1 uses sparse slots (0...13, 15...19, 22...26))
 -

Deployment Sim / Sim2Sim

- This command helps us confirm the fix with the R1 actuator parameters.

```
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ python - <<'PY'
import mujoco
from mjlab.entity.entity import Entity
from src.assets.robots.unitree_r1.rl_constants import get_r1_robot_cfg

robot = Entity(get_r1_robot_cfg())
model = robot.spec.compile()

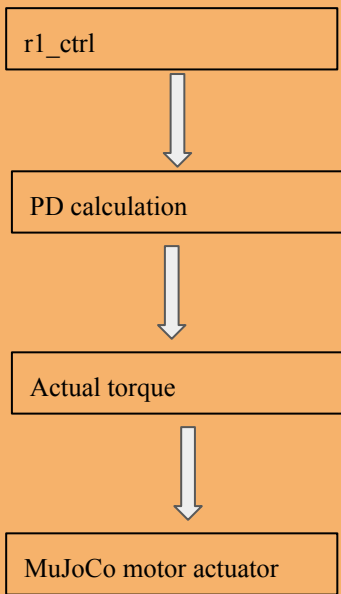
print("Number of actuators:", model.nu)
print("\nActuator order:")

for i in range(model.nu):
    name = mujoco.mj_id2name(
        model,
        mujoco.mjtObj.mjOBJ_ACTUATOR,
        i
    )
    print(f"{i:2d}: {name}")

PY
Number of actuators: 24

Actuator order:
0: left_hip_pitch_joint
1: left_hip_roll_joint
2: left_hip_yaw_joint
3: left_knee_joint
4: right_hip_pitch_joint
5: right_hip_roll_joint
6: right_hip_yaw_joint
7: right_knee_joint
8: left_ankle_pitch_joint
9: left_ankle_roll_joint
10: right_ankle_pitch_joint
11: right_ankle_roll_joint
12: waist_roll_joint
13: waist_yaw_joint
14: left_shoulder_pitch_joint
15: left_shoulder_roll_joint
16: right_shoulder_pitch_joint
17: right_shoulder_roll_joint
18: left_shoulder_yaw_joint
19: left_elbow_joint
20: left_wrist_roll_joint
21: right_shoulder_yaw_joint
22: right_elbow_joint
23: right_wrist_roll_joint
```

Physical Deployment Architecture



- From reviewing files, it was found that the the both the Unitree's deployment bridge and MJLab contains PD actuators. So we removed MJLab's PD actuator and solely use the MuJoCo actuators.

R1 DDS Mapping Problem

- For other robots (like the G1), the MuJoCo actuator index is the same as the DDS motor index.
- However, for the R1, it's different; it utilizes sparse Unitree motor slots:

0,1,2,3,4,5,
6,7,8,9,10,11,
12,13,
15,16,17,18,19,
22,23,24,25,26

- We needed to map the correct motor slots to the MuJoCo actuator index

unitree_sdk2_bridge.h

- An R1-specific mapping function was added onto **unitree_sdk2_bridge.h**, which follows the motor_id instead of the index.

Standalone R1 deployment Scene

- `src/assets/robots/unitree_r1/xmls/scene_r1.xml` was made for the R1 model, replacing the MJLAB PD actuators with 24 torque `<motor>` actuators while preserving their torque limits.
- Similar to the G1 scene, a ground plane and lighting was implemented.

Sensor Layout based on Unitree's bridge

- The simulator bridge assumes joint positions, joint velocities, and joint actuator forces.
- We generated exactly 24 jointpos sensors, jointvel sensors, and jointactuatorfrc sensors.
 - Next, we added variables such as imu_quat, imu_gyro (R1 based parameters).
 - This is so we have the C++ bridge and MuJoCo scene match in terms of their actuators, sensors, and sensor-data values.

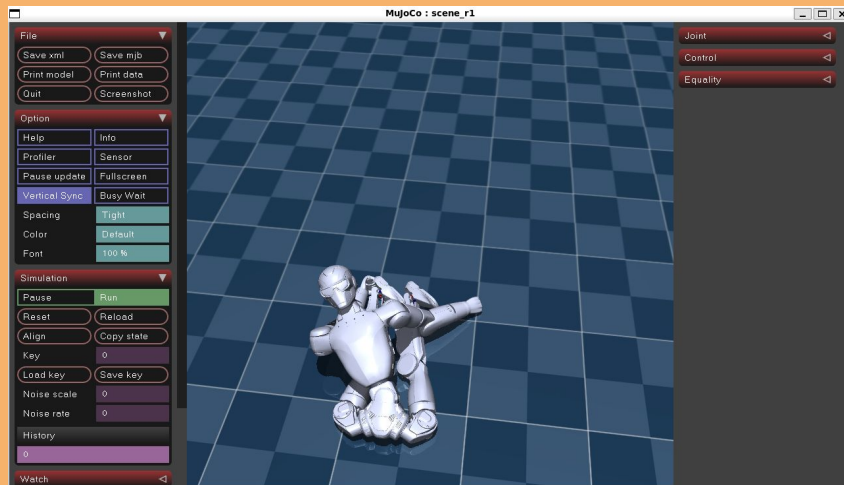
r1_ctrl mapping bug

- Another issue related to mapping occurs in **r1_ctrl** (the R1 controller) related to using the index values rather than the sparse MuJoCo actuator order.

Configuring unitree_mujoco for the R1

The unitree_mujoco simulator was configured for the g1 robot. We set it up for the R1, implementing its robot_scene and robot id.

Finally, we launch the Sim2Sim system, loading the R1 and printing the 24 actuators, 77 sensors, and 88 sensor-data values.



```
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab/deploy/robots/r1/build$ ./r1_ctrl --network=lo
--- Unitree Robotics ---
R1 Controller
1788543584.536091 [0] r1_ctrl: selected interface "lo" is not multicast-capable: disabling multicast
[2026-09-04 13:39:44.840] [info] Waiting for connection to robot...
[2026-09-04 13:39:45.040] [info] Connected to robot.
[2026-09-04 13:39:45.141] [info] Initializing State_Passive ...
[2026-09-04 13:39:45.141] [info] Initializing State_FixStand ...
[2026-09-04 13:39:45.141] [info] Initializing State_Velocity ...
[2026-09-04 13:39:45.142] [info] Policy directory: /home/mario/unitree_rl_mjlab/deploy/robots/r1/config/policy/velocity/v0
[2026-09-04 13:39:45.253] [info] FSM: Start Passive
Press [L2 + Up] to enter FixStand mode.
And then press [R1 + X] to start controlling the robot.
^C
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab/deploy/robots/r1/build$
```

Configuring unitree_mujoco for the R1

```
mario@LAPTOP-9NUH3CE9: ~  
(base) mario@LAPTOP-9NUH3CE9:~$ conda activate unitree_rl_mjlab  
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~$ cd ~/unitree_rl_mjlab  
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/unitree_rl_mjlab$ MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA \\\n./simulate/build/unitree_mujoco  
MuJoCo version 3.3.6  
MuJoCo data is prepared  
1788543481.495968 [0] unitree_mu: selected interface "lo" is not multicast-capable: disabling multicast  
<<----- Link ----->>  
Link_index: 0, name: world  
Link_index: 1, name: pelvis  
Link_index: 2, name: left_hip_pitch_link  
Link_index: 3, name: left_hip_roll_link  
Link_index: 4, name: left_hip_yaw_link  
Link_index: 5, name: left_knee_link  
Link_index: 6, name: left_ankle_pitch_link  
Link_index: 7, name: left_ankle_roll_link  
Link_index: 8, name: right_hip_pitch_link  
Link_index: 9, name: right_hip_roll_link  
Link_index: 10, name: right_hip_yaw_link  
Link_index: 11, name: right_knee_link  
Link_index: 12, name: right_ankle_pitch_link  
Link_index: 13, name: right_ankle_roll_link  
Link_index: 14, name: waist_roll_link  
Link_index: 15, name: torso_link  
Link_index: 16, name: left_shoulder_pitch_link  
Link_index: 17, name: left_shoulder_roll_link  
Link_index: 18, name: left_shoulder_yaw_link  
Link_index: 19, name: left_elbow_link  
Link_index: 20, name: left_wrist_roll_link  
Link_index: 21, name: right_shoulder_pitch_link  
Link_index: 22, name: right_shoulder_roll_link  
Link_index: 23, name: right_shoulder_yaw_link  
Link_index: 24, name: right_elbow_link  
Link_index: 25, name: right_wrist_roll_link  
<<----- Joint ----->>  
Joint_index: 0, name: Floating_base_joint  
Joint_index: 1, name: left_hip_pitch_joint  
Joint_index: 2, name: left_hip_roll_joint  
Joint_index: 3, name: left_hip_yaw_joint  
Joint_index: 4, name: left_knee_joint  
Joint_index: 5, name: left_ankle_pitch_joint  
Joint_index: 6, name: left_ankle_roll_joint  
Joint_index: 7, name: right_hip_pitch_joint  
Joint_index: 8, name: right_hip_roll_joint  
Joint_index: 9, name: right_hip_yaw_joint  
Joint_index: 10, name: right_knee_joint  
Joint_index: 11, name: right_ankle_pitch_joint  
Joint_index: 12, name: right_ankle_roll_joint  
Joint_index: 13, name: waist_roll_joint  
Joint_index: 14, name: waist_yaw_joint
```

```
mario@LAPTOP-9NUH3CE9: ~  
Joint_index: 15, name: left_shoulder_pitch_joint  
Joint_index: 16, name: left_shoulder_roll_joint  
Joint_index: 17, name: left_shoulder_yaw_joint  
Joint_index: 18, name: left_elbow_joint  
Joint_index: 19, name: left_wrist_roll_joint  
Joint_index: 20, name: right_shoulder_pitch_joint  
Joint_index: 21, name: right_shoulder_roll_joint  
Joint_index: 22, name: right_shoulder_yaw_joint  
Joint_index: 23, name: right_elbow_joint  
Joint_index: 24, name: right_wrist_roll_joint  
<<----- Actuator ----->>  
Actuator_index: 0, name: left_hip_pitch_joint  
Actuator_index: 1, name: left_hip_roll_joint  
Actuator_index: 2, name: left_hip_yaw_joint  
Actuator_index: 3, name: left_knee_joint  
Actuator_index: 4, name: right_hip_pitch_joint  
Actuator_index: 5, name: right_hip_roll_joint  
Actuator_index: 6, name: right_hip_yaw_joint  
Actuator_index: 7, name: right_knee_joint  
Actuator_index: 8, name: left_ankle_pitch_joint  
Actuator_index: 9, name: left_ankle_roll_joint  
Actuator_index: 10, name: right_ankle_pitch_joint  
Actuator_index: 11, name: right_ankle_roll_joint  
Actuator_index: 12, name: waist_roll_joint  
Actuator_index: 13, name: waist_yaw_joint  
Actuator_index: 14, name: left_shoulder_pitch_joint  
Actuator_index: 15, name: left_shoulder_roll_joint  
Actuator_index: 16, name: right_shoulder_pitch_joint  
Actuator_index: 17, name: right_shoulder_roll_joint  
Actuator_index: 18, name: left_shoulder_yaw_joint  
Actuator_index: 19, name: left_elbow_joint  
Actuator_index: 20, name: left_wrist_roll_joint  
Actuator_index: 21, name: right_shoulder_yaw_joint  
Actuator_index: 22, name: right_elbow_joint  
Actuator_index: 23, name: right_wrist_roll_joint  
<<----- Sensor ----->>  
Sensor_index: 0, name: left_hip_pitch_joint_pos, dim: 1  
Sensor_index: 1, name: left_hip_roll_joint_pos, dim: 1  
Sensor_index: 2, name: left_hip_yaw_joint_pos, dim: 1  
Sensor_index: 3, name: left_knee_joint_pos, dim: 1  
Sensor_index: 4, name: right_hip_pitch_joint_pos, dim: 1  
Sensor_index: 5, name: right_hip_roll_joint_pos, dim: 1  
Sensor_index: 6, name: right_hip_yaw_joint_pos, dim: 1  
Sensor_index: 7, name: right_knee_joint_pos, dim: 1  
Sensor_index: 8, name: left_ankle_pitch_joint_pos, dim: 1  
Sensor_index: 9, name: left_ankle_roll_joint_pos, dim: 1  
Sensor_index: 10, name: right_ankle_pitch_joint_pos, dim: 1  
Sensor_index: 11, name: right_ankle_roll_joint_pos, dim: 1  
Sensor_index: 12, name: waist_roll_joint_pos, dim: 1
```

```
mario@LAPTOP-9NUH3CE9: ~  
Sensor_index: 13, name: waist_yaw_joint_pos, dim: 1  
Sensor_index: 14, name: left_shoulder_pitch_joint_pos, dim: 1  
Sensor_index: 15, name: left_shoulder_roll_joint_pos, dim: 1  
Sensor_index: 16, name: right_shoulder_pitch_joint_pos, dim: 1  
Sensor_index: 17, name: right_shoulder_roll_joint_pos, dim: 1  
Sensor_index: 18, name: left_shoulder_yaw_joint_pos, dim: 1  
Sensor_index: 19, name: left_elbow_joint_pos, dim: 1  
Sensor_index: 20, name: left_wrist_roll_joint_pos, dim: 1  
Sensor_index: 21, name: right_shoulder_yaw_joint_pos, dim: 1  
Sensor_index: 22, name: right_elbow_joint_pos, dim: 1  
Sensor_index: 23, name: right_wrist_roll_joint_pos, dim: 1  
Sensor_index: 24, name: left_hip_pitch_joint_vel, dim: 1  
Sensor_index: 25, name: left_hip_roll_joint_vel, dim: 1  
Sensor_index: 26, name: left_hip_yaw_joint_vel, dim: 1  
Sensor_index: 27, name: left_knee_joint_vel, dim: 1  
Sensor_index: 28, name: right_hip_pitch_joint_vel, dim: 1  
Sensor_index: 29, name: right_hip_roll_joint_vel, dim: 1  
Sensor_index: 30, name: right_hip_yaw_joint_vel, dim: 1  
Sensor_index: 31, name: right_knee_joint_vel, dim: 1  
Sensor_index: 32, name: left_ankle_pitch_joint_vel, dim: 1  
Sensor_index: 33, name: left_ankle_roll_joint_vel, dim: 1  
Sensor_index: 34, name: right_ankle_pitch_joint_vel, dim: 1  
Sensor_index: 35, name: right_ankle_roll_joint_vel, dim: 1  
Sensor_index: 36, name: waist_roll_joint_vel, dim: 1  
Sensor_index: 37, name: waist_yaw_joint_vel, dim: 1  
Sensor_index: 38, name: left_shoulder_pitch_joint_vel, dim: 1  
Sensor_index: 39, name: left_shoulder_roll_joint_vel, dim: 1  
Sensor_index: 40, name: right_shoulder_pitch_joint_vel, dim: 1  
Sensor_index: 41, name: right_shoulder_roll_joint_vel, dim: 1  
Sensor_index: 42, name: left_shoulder_yaw_joint_vel, dim: 1  
Sensor_index: 43, name: left_elbow_joint_vel, dim: 1  
Sensor_index: 44, name: left_wrist_roll_joint_vel, dim: 1  
Sensor_index: 45, name: right_shoulder_yaw_joint_vel, dim: 1  
Sensor_index: 46, name: right_elbow_joint_vel, dim: 1  
Sensor_index: 47, name: right_wrist_roll_joint_vel, dim: 1  
Sensor_index: 48, name: left_hip_pitch_joint_torque, dim: 1  
Sensor_index: 49, name: left_hip_roll_joint_torque, dim: 1  
Sensor_index: 50, name: left_hip_yaw_joint_torque, dim: 1  
Sensor_index: 51, name: left_knee_joint_torque, dim: 1  
Sensor_index: 52, name: right_hip_pitch_joint_torque, dim: 1  
Sensor_index: 53, name: right_hip_roll_joint_torque, dim: 1  
Sensor_index: 54, name: right_hip_yaw_joint_torque, dim: 1  
Sensor_index: 55, name: right_knee_joint_torque, dim: 1  
Sensor_index: 56, name: left_ankle_pitch_joint_torque, dim: 1  
Sensor_index: 57, name: left_ankle_roll_joint_torque, dim: 1  
Sensor_index: 58, name: right_ankle_pitch_joint_torque, dim: 1  
Sensor_index: 59, name: right_ankle_roll_joint_torque, dim: 1  
Sensor_index: 60, name: waist_roll_joint_torque, dim: 1  
Sensor_index: 61, name: waist_yaw_joint_torque, dim: 1  
Sensor_index: 62, name: left_shoulder_pitch_joint_torque, dim: 1  
Sensor_index: 63, name: left_shoulder_roll_joint_torque, dim: 1
```

References

- https://github.com/unitreerobotics/unitree_rl_mjlab
- https://github.com/unitreerobotics/unitree_rl_mjlab/blob/main/doc/setup_en.md